UNIVERSIDAD RAFAEL LANDÍVAR
INGENIERÍA EN INFORMÁTICA Y SISTEMAS
ESTRUCTURA DE DATOS II
ING. MANUEL ROJAS
CUARTO SEMESTRE
PROYECTO II
TAY RUIZ JORGE ERNESTO 1532625
SOTO TZUL JONATHAN ANDRÉ 1533825
QUIJIVIX SUCHI PABLO DAVID 1578125

Universidad Rafael Landívar
Tradición Jesuita en Guatemala

Manual técnico — CampusLink (Red Social Universitaria)

Proyecto II

Quetzaltenango 05 noviembre de 2026

Índice

Diagrama de flujo 1

..... 1

Diagrama de Clases 2

Explicación de la implementación 3

- Organización del proyecto 3
- Arquitectura por capas 4
- Modelos de datos 4
- Estructuras auxiliares 4
- Tabla Hash (HashTable) 5
- Trie (Trie) 5
- Grafo (Graph) 5
- Union-Find (UnionFind) 6
- Cálculo de afinidad (AC) 6
- Servicio coordinador (SNS) 6
- Persistencia (ChargeCSV) 7
- Interfaz gráfica 7
- Consideraciones y limitaciones 8

Análisis de complejidad (Big-O) 9

- Estructuras auxiliares 9
- Tabla Hash 10
- Trie 10
- Union-Find 11
- Grafo 11
- Servicio de afinidad y persistencia 12
- Operaciones del servicio SNS y de la interfaz 13

Justificación de la elección de las estructuras de datos 14

- Tabla Hash con encadenamiento separado 14
- Trie 14
- Grafo no dirigido y ponderado con lista de adyacencia 15
- Union-Find (conjuntos disjuntos) 15
- Montículo mínimo (MinMound) 15

Lista enlazada simple (LinkedList<T> propia)..... 15
Índice de llaves (IndiceLlaves)..... 16
Arreglos con duplicación de capacidad..... 16
Cola (Queue<int>) en el recorrido BFS..... 16
Archivos CSV..... 16

Diagrama de flujo

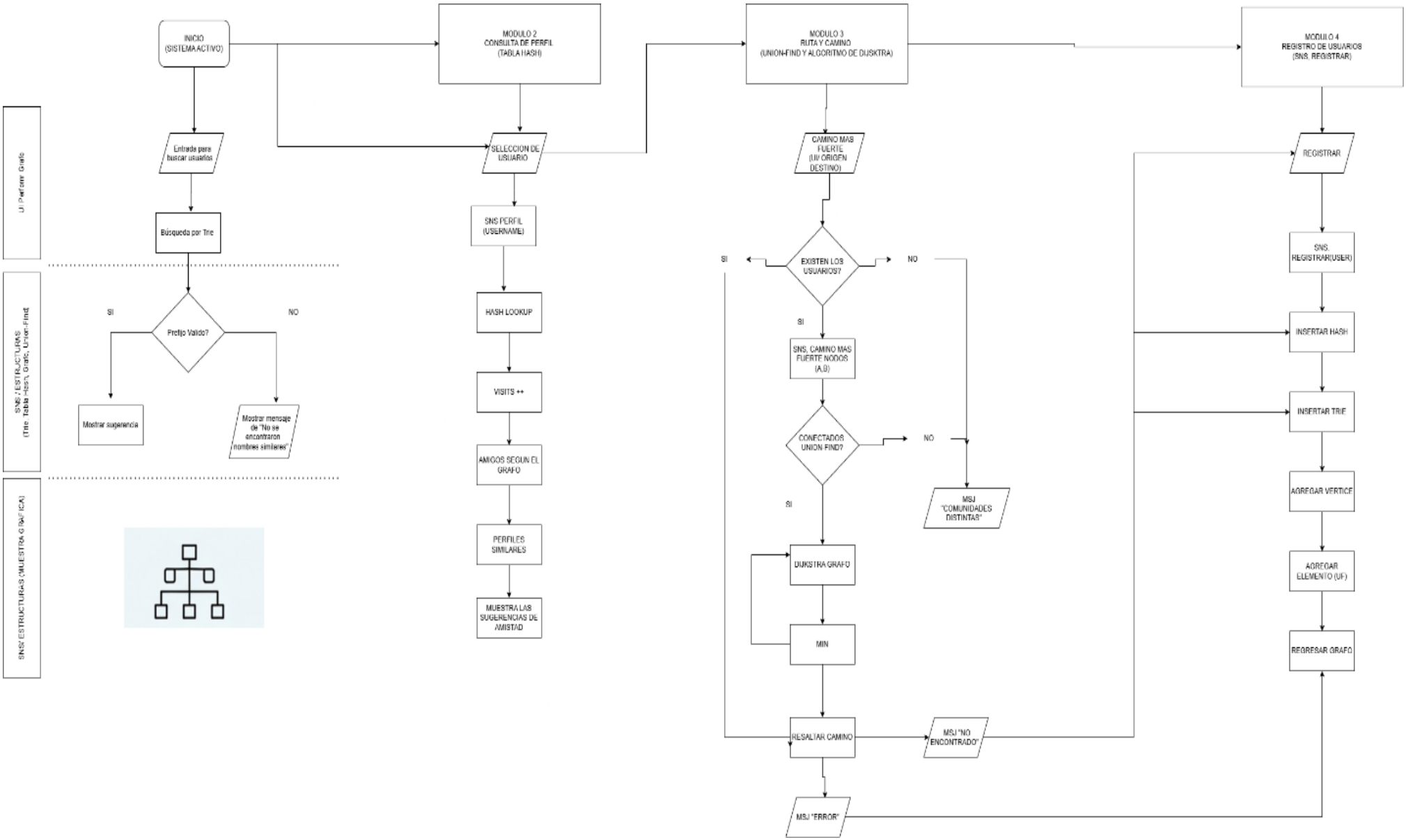
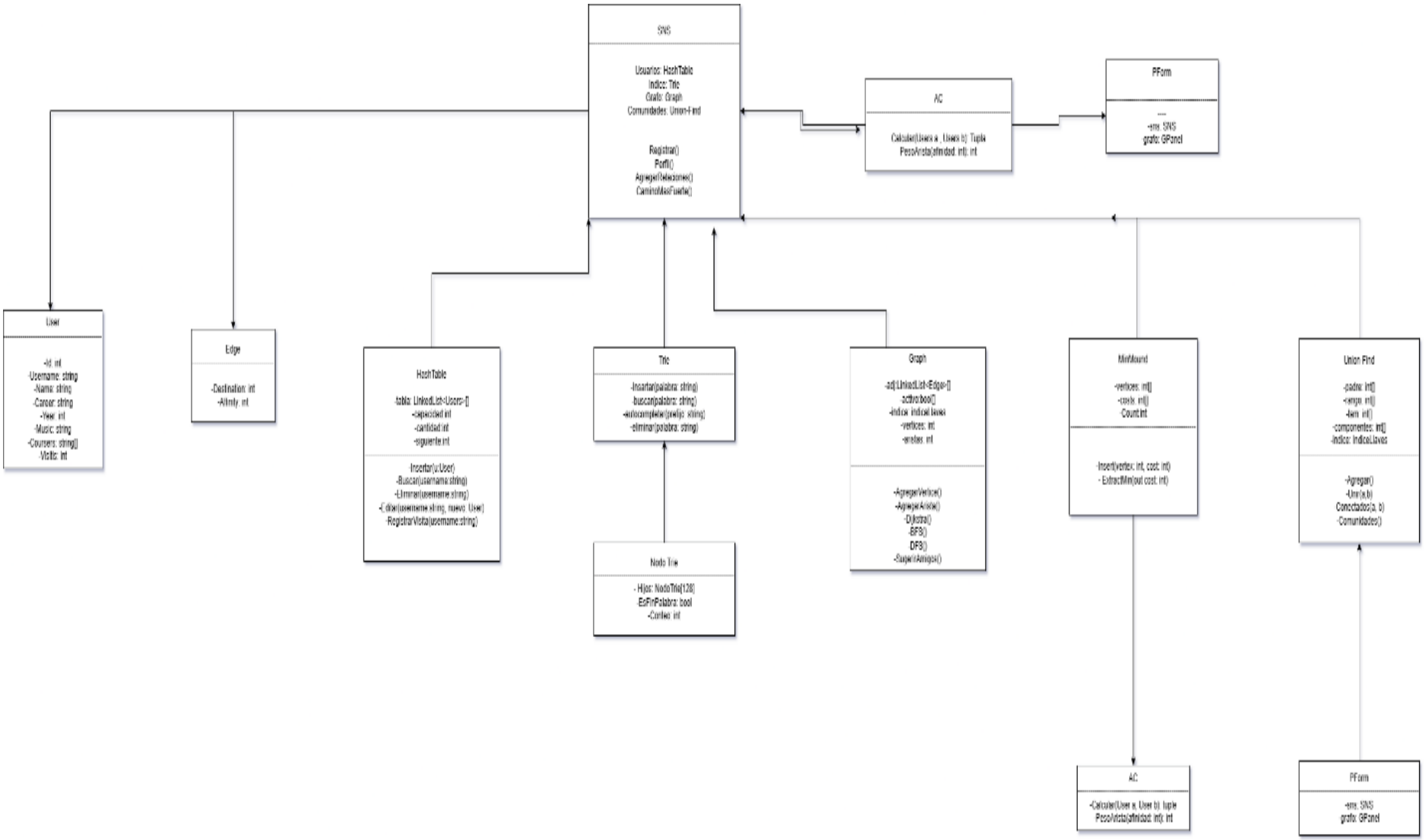


Diagrama de Clases



Explicación de la implementación

CampusLink es una aplicación de escritorio para Windows, desarrollada en C# sobre .NET 8 con Windows Forms (net8.0-windows), que modela una red social universitaria. Su núcleo lo forman cuatro estructuras de datos implementadas desde cero (Tabla Hash, Trie, Grafo y Union-Find), coordinadas por una capa de servicio que las mantiene consistentes entre sí.

Organización del proyecto

Ubicación	Archivos	Responsabilidad
Raíz	Program.cs, USN.csproj	Punto de entrada: abre PForm; con el argumento --trials abre el menú de pruebas por consola. Configuración del proyecto.
Models/	User.cs, Edge.cs	Registro de usuario y arista del grafo.
Structures/	HashTable.cs, Trie.cs, Graph.cs, union_find.cs	Las cuatro estructuras principales.
Structures/Auxiliar/	linkedlist.cs, m_mound.cs, Util.cs	Lista enlazada propia, montículo mínimo, función hash, normalización de llaves e IndiceLlaves.
Services/	SNS.cs, AC.cs, ChargeCSV.cs	Orquestación de las estructuras, cálculo de afinidad y persistencia CSV.
UI/	PForm.cs, GPanel.cs, Assets/Dialogocampos.cs, Assets/Tema.cs	Ventana principal, dibujo del grafo, diálogo de captura de datos y tema visual.
Data/	user.csv, relations.csv	Archivos de datos que se cargan al iniciar.
Trials/	trial.cs	Menú de pruebas por consola (opciones 1 a 15 y salida con 0).

El código establece como restricción no usar List<T>, LinkedList<T>, Dictionary, HashSet, PriorityQueue, SortedSet ni LINQ en las estructuras. Por ello se implementaron LinkedList<T> y MinMound propios y se trabaja con arreglos T[]. La única colección de la biblioteca estándar usada en la capa de estructuras es Queue<int>, exclusivamente en el recorrido BFS. El Dictionary aparece solo en GPanel, que pertenece a la capa de dibujo.

Arquitectura por capas

La implementación se organiza en tres capas. La interfaz (PForm, GPanel, DialogoCampos, Tema) captura datos y muestra resultados. Los servicios (SNS, AC, ChargeCSV) contienen la lógica del sistema. Las estructuras y los modelos almacenan los datos y ejecutan los algoritmos.

La regla central es que la interfaz solo se comunica con SNS, y toda operación que toca más de una estructura vive en SNS, para que ninguna quede desfasada de las demás. SNS expone cuatro estructuras:

Campo de SNS	Estructura	Responsabilidad
Usuarios	HashTable	Fuente de verdad: el User completo, localizado por Username (llave primaria).
Indice	Trie	Autocompletado de usernames; solo guarda llaves.
Grafo	Graph	Relaciones ponderadas por afinidad; BFS, DFS, Dijkstra y sugerencias de amistad.
Comunidades	UnionFind	Comunidades; valida la conectividad antes de ejecutar Dijkstra.

Modelos de datos

User contiene Id, Username, Name, Career, Year, Music, Courses (arreglo de cadenas) y Visits. El Username es la llave primaria de todas las estructuras y se normaliza siempre con Util.Norm (recorte de espacios y minúsculas invariantes). El Id lo asigna la tabla hash al insertar y no se reutiliza aunque el usuario se elimine. Las relaciones no se guardan en User: pertenecen al grafo y referencian a los usuarios por Username.

Edge contiene Destination (índice interno del vértice destino) y Affinity (valor de 0 a 100). Es una clase y no una estructura (struct) para que LinkedList<Edge>.Find devuelva null cuando no encuentra el elemento.

Estructuras auxiliares

- **LinkedList<T> propia.** Lista simple con referencias head y tail y contador Count. Implementa AddLast, RemoveFirst, Find(Predicate), Remove(Predicate), ToArray y Clear. Remove contempla tres casos: eliminar la cabeza, un nodo intermedio o la cola. Se usa como cadena de colisiones en la tabla hash y como lista de adyacencia en el grafo.
- **MinMound.** Montículo mínimo en dos arreglos paralelos, vertices y costs. Insert agrega al final y sube el elemento (HeapifyUp); ExtractMin devuelve la raíz, mueve el último elemento a la posición 0 y lo hunde (HeapifyDown). La capacidad inicial es 16 y se duplica cuando se llena. Lo usa Dijkstra.
- **Util.Hash.** Variante de djb2 con XOR: parte de 5381 y, por cada carácter, calcula h = (h * 33) ^ c; el resultado se limita a valores no negativos con la máscara 0x7FFFFFFF.
- **IndiceLlaves.** Mapa propio de llave (cadena) a índice entero consecutivo (0, 1, 2…). Usa direccionamiento abierto con sondeo lineal sobre un arreglo de enteros (-1 indica casilla vacía) y un arreglo de nombres. Crece al duplicar su tamaño cuando el factor de

ocupación superaría 0.5. No soporta borrado. Lo usan `Graph` y `UnionFind` para trabajar con arreglos de enteros en lugar de diccionarios.

Tabla Hash (`HashTable`)

Usa encadenamiento separado: cada casilla es una `LinkedList<User>`. La capacidad inicial es 16 (mínimo 8) y la posición se calcula como `Util.Hash(username) % capacidad`. Antes de insertar, si el factor de carga resultante superaría 0.75, la tabla duplica su capacidad y redistribuye todos los registros.

- `Insertar` normaliza la llave, rechaza duplicados sin sobrescribir, asigna `Id = siguienteId++` y devuelve si insertó, el índice, el largo de la cadena y si hubo redimensionamiento.
- `Buscar`, `Existe`, `Eliminar` y `RegistrarVisita` localizan el registro por llave. Cada consulta de perfil incrementa el contador de visitas.
- `Editar` reemplaza los datos conservando la llave primaria y el `Id`.
- `ObtenerTodos` devuelve todos los usuarios en un arreglo.

Trie (`Trie`)

Cada nodo (`NodoTrie`) tiene un arreglo de 128 hijos (códigos ASCII), la bandera `EsFinPalabra` y `Conteo`, la cantidad de palabras de su subárbol. Solo guarda usernames; el perfil completo se pide después a la tabla hash. Las llaves con caracteres no ASCII (tildes, ñ) se rechazan.

- `Insertar` crea los nodos faltantes e incrementa `Conteo` a lo largo del camino.
- `Buscar` indica si la palabra existe y si es prefijo de alguna otra.
- `Autocompletar(prefijo, max)` ubica el nodo del prefijo y recorre el subárbol en orden de código ASCII, de modo que las sugerencias salen en orden alfabético; se detiene al reunir `max` resultados y devuelve además el total de coincidencias (`Conteo` del nodo).
- `Eliminar` decrementa `Conteo` en el camino y poda las ramas que quedan sin palabras.

Grafo (`Graph`)

Grafo no dirigido y ponderado, con lista de adyacencia: un arreglo de `LinkedList<Edge>` indexado por el entero que entrega `IndiceLlaves`, más un arreglo `activo[]` que marca los vértices vigentes (borrado lógico). Ambos arreglos se duplican al llenarse.

- **Aristas.** `AgregarArista` limita la afinidad al rango 0 a 100, rechaza lazos y vértices inexistentes, y crea dos aristas simétricas. Si la relación ya existía, solo actualiza el peso en ambos sentidos.
- **Costo.** Dijkstra usa `Costo(afinidad) = 101 − AC.PesoArista(afinidad)`. El costo siempre es positivo (de 1 a 101) y, a mayor afinidad, menor costo, por lo que el camino más corto es el más afín.
- **Eliminación.** `EliminarVertice` quita las aristas de sus vecinos, vacía su lista y lo marca inactivo. La llave permanece en `IndiceLlaves`; si el username se registra de nuevo, el vértice se reactiva.
- **BFS.** Recorrido por niveles con `Queue<int>`; devuelve el orden de visita y el nivel (saltos desde el origen) de cada vértice alcanzado.
- **DFS.** Recorrido en profundidad en preorden, implementado con recursión; devuelve el orden de visita y la profundidad.

- **Dijkstra.** Mantiene los arreglos dist, previo y listo, y un MinMound con borrado perezoso: las entradas obsoletas se descartan al extraerlas. Se detiene al extraer el destino. Reconstruye el camino con previo y calcula el costo total (dist del destino) y la afinidad promedio (suma de afinidades de las aristas del camino entre el número de saltos, división entera). Si origen y destino coinciden, devuelve un camino de un solo vértice con costo 0.
- **Sugerencias de amistad (SugerirAmigos).** Si el usuario tiene amigos directos, evalúa los amigos de sus amigos: el puntaje de cada candidato es la suma, sobre los amigos en común, del mínimo entre las dos afinidades del camino. Se excluyen el propio usuario y sus amigos directos. Si no tiene conexiones, ordena a los usuarios por cantidad de conexiones. Devuelve los max mejores con puntaje mayor que cero (la interfaz pide 5).

Union-Find (UnionFind)

Conjuntos disjuntos sobre usernames, con los arreglos padre, rango y tam (capacidad inicial 16, se duplican) e IndiceLlaves para convertir llaves en índices. Raiz aplica compresión de caminos de forma iterativa, en dos pasadas y sin recursión. Unir aplica unión por rango y actualiza el tamaño y el número de componentes. Conectados valida si dos usuarios están en la misma comunidad, y Comunidades agrupa todos los usernames por raíz. Union-Find no permite deshacer uniones, por lo que Reiniciar deja la estructura vacía para reconstruirla desde el grafo.

Cálculo de afinidad (AC)

AC.Calcular asigna hasta 100 puntos con cuatro criterios de 25 puntos cada uno: misma música, misma carrera, mismo año (el año debe ser distinto de 0) y cursos en común. Las comparaciones de texto ignoran mayúsculas. Los cursos puntúan de forma proporcional: 25 × cursos_en_común / máximo(cursos de A, cursos de B) con división entera. También devuelve qué criterios se cumplen, dato que la interfaz usa para colorear las aristas. AC.PesoArista(afinidad, distancia = 1) aplica (afinidad / 100) × (1 / distancia) × 100, redondeado al entero; con distancia 1, el peso es igual a la afinidad.

Servicio coordinador (SNS)

Operación	Estructuras que intervienen y orden
Registrar	HashTable → Trie → Graph → UnionFind. Si el Trie rechaza la llave (caracteres no ASCII), se revierte el alta en la tabla hash para no dejar un usuario invisible al buscador.
Buscar	Trie.Autocompletar: sugerencias mientras se escribe.
Perfil	HashTable.Buscar y RegistrarVisita: cada consulta cuenta como una visita.
AgregarRelacion	AC.Calcular → Graph.AgregarArista → UnionFind.Unir. La unión se realiza siempre que la arista se crea.
CaminoMasFuerte	HashTable verifica que existan ambos usuarios → UnionFind.Conectados valida la comunidad → Graph.Dijkstra.

Operación	Estructuras que intervienen y orden
SugerenciasDeAmistad	Graph.SugerirAmigos.
Recorrer	Graph.BFS y Graph.DFS desde el usuario indicado.
ObtenerComunidades	UnionFind.Comunidades.
Eliminar	HashTable.Eliminar → Trie.Eliminar → Graph.EliminarVertice → Reconstruir.
EliminarRelacion	Graph.EliminarArista → Reconstruir.
EditarUsuario	Reemplaza el perfil y recalcula la afinidad de todas sus relaciones.
Cargar / Guardar	ChargeCSV: lectura y escritura de los archivos de usuarios y relaciones.

Persistencia (ChargeCSV)

Solo se guardan los datos fuente (usuarios y relaciones). El Trie y el UnionFind son índices derivados y se reconstruyen con Reconstruir.

- user.csv tiene las columnas username,name,career,year,music,courses,visits, con los cursos separados por ;.
- relations.csv tiene las columnas user1,user2,affinity. Al cargar, la afinidad es opcional: si falta, se calcula con AC a partir de los dos perfiles.
- Al guardar, las comas y los saltos de línea de los textos se reemplazan por espacios. Al cargar, el encabezado se reconoce por la línea exacta de columnas, para no confundir un usuario llamado user1 con el encabezado.
- CargarUsuarios informa cuántas filas se cargaron, cuántas eran duplicadas y cuántas inválidas. CargarRelaciones omite las filas cuyo extremo no existe en la tabla hash.
- Reconstruir reinicia UnionFind y recorre la tabla hash para insertar cada username en el Trie, el grafo y UnionFind; después une los extremos de todas las aristas del grafo.

Interfaz gráfica

PForm es la ventana principal. Al iniciar intenta cargar Data/user.csv y Data/relations.csv y, si no existen, arranca vacía. Se compone de un menú lateral de 10 opciones, un buscador con autocompletado, tres columnas (perfil, grafo y paneles de camino y sugerencias) y una barra de estado.

Opción del menú	Método	Acción sobre el sistema
Buscar usuario	txtBuscar.Focus	Lleva el cursor al buscador.
Registrar usuario	AccionRegistrar	Pide seis campos y llama a SNS.Registrar.

Opción del menú	Método	Acción sobre el sistema
Agregar relación	AccionRelacion	Pide dos usernames y llama a SNS.AgregarRelacion.
Sugerencias de amistad	txtOrigen.Focus	Lleva el cursor al campo Origen del panel de camino.
Comunidades	AccionComunidades	Llama a SNS.ObtenerComunidades, colorea los nodos y muestra el listado.
Recorridos BFS / DFS	AccionRecorrido	Pide usuario inicial y tipo de recorrido; llama a SNS.Recorrer.
Ver estructuras	AccionVerEstructuras	Muestra el contenido de la tabla hash, el trie, Union-Find y el grafo.
Cargar datos (CSV / TXT)	AccionCargar	Abre dos selectores de archivo y reemplaza los datos actuales.
Guardar datos (CSV)	AccionGuardar	Llama a SNS.Guardar.
Eliminar usuario	AccionEliminar	Pide el username, confirma y llama a SNS.Eliminar.

El autocompletado consulta el Trie con cada tecla y muestra hasta 7 sugerencias en una lista flotante. Al presionar Enter o hacer clic en una sugerencia se abre el perfil, que se obtiene de la tabla hash y se acompaña de las sugerencias de amistad del usuario.

GPanel dibuja el grafo con GDI+ y no conoce las estructuras: recibe la lista de usuarios y una función que devuelve los vecinos de un username. Los nodos se distribuyen en una circunferencia según su Id; cada criterio de afinidad compartido se dibuja como una línea paralela de color (gris para música, verde para carrera, azul para cursos y naranja para año); el camino de Dijkstra se resalta con línea discontinua; las comunidades se pintan con una paleta de 8 colores; y un clic sobre un nodo (radio de 24 px) dispara el evento NodoClic, que abre el perfil. DialogoCampos es un diálogo modal genérico que solicita varios textos, y Tema concentra colores, fuentes y el botón base.

Consideraciones y limitaciones

- El Trie solo admite usernames ASCII; los demás se rechazan al registrar.
- Union-Find no deshace uniones, por lo que eliminar un usuario o una relación obliga a reconstruirlo desde el grafo.
- IndiceLlaves no elimina llaves: los vértices eliminados permanecen en el índice como inactivos y los arreglos de BFS, DFS y Dijkstra se dimensionan con ese total.
- El DFS es recursivo, por lo que la profundidad de la pila crece con la longitud del camino explorado.
- EditarUsuario y EliminarRelacion existen en SNS y en el menú de consola, pero el menú de la interfaz gráfica no los expone.

- `Cargar` crea una instancia nueva de `SNS` con los archivos elegidos y `Guardar` escribe de vuelta en esos mismos archivos. Si al cargar se omite el archivo de relaciones, el guardado de relaciones no dispone de una ruta válida y falla.

Análisis de complejidad (Big-O)

El análisis se realiza sobre el código del sistema y distingue el caso promedio del peor caso cuando la diferencia es relevante. Se emplea la siguiente notación:

Símbolo	Significado
n	Cantidad de usuarios almacenados en la tabla hash.
V	Cantidad de vértices registrados en el índice del grafo (puede incluir vértices eliminados lógicamente, por lo que V ≥ n).
E	Cantidad de relaciones (aristas no dirigidas) del grafo.
d(u)	Grado del vértice u, es decir, el largo de su lista de adyacencia.
L	Longitud del username. Lmax es la longitud del username más largo.
Σ	Tamaño del alfabeto del Trie (128 caracteres ASCII).
M	Cantidad máxima de sugerencias solicitadas al autocompletado.
α(n)	Inversa de la función de Ackermann; para cualquier valor práctico de n es menor o igual que 4.

Todas las llaves se normalizan y se procesan carácter por carácter (`Util.Norm` y `Util.Hash` cuestan O(L)), por lo que L aparece de forma explícita. Para usernames de longitud acotada, L se comporta como una constante.

Estructuras auxiliares

Estructura y operación	Tiempo	Observación
`LinkedList<T>.AddLast`	O(1)	Se apoya en la referencia `tail`.
`LinkedList<T>.RemoveFirst`, `Clear`	O(1)	Solo modifican referencias.
`LinkedList<T>.Find`, `Remove`, `ToArray`	O(k)	k = cantidad de elementos de la lista; recorren desde `head`.
`MinMound.Insert`	O(log m)	m = elementos del montículo; la duplicación de capacidad cuesta O(m) y es O(1) amortizado.
`MinMound.ExtractMin`	O(log m)	`HeapifyDown` desciende como máximo la altura del árbol.
`IndiceLlaves.Buscar`, `Agregar`	O(L) esperado; O(n·L) en	El factor de ocupación se mantiene en 0.5 o menos, por lo que el sondeo lineal requiere pocas casillas en promedio. El crecimiento

Estructura y operación	Tiempo	Observación
	el peor caso	(Crecer) cuesta O(n·L) y es O(L) amortizado por inserción.

Espacio: LinkedList<T> O(k), MinMound O(m) e IndiceLlaves O(n).

Tabla Hash

Con la función Util.Hash y un factor de carga limitado a 0.75, la longitud esperada de cada cadena es una constante menor que 1. Por ello el costo esperado está dominado por el cálculo del hash y la comparación de llaves, O(L). En el peor caso (todas las llaves en la misma casilla) la cadena contiene n elementos.

Operación	Caso promedio	Peor caso	Observación
Insertar	O(L)	O(n·L)	Incluye la búsqueda de duplicados en la cadena.
Redimensionar	O(n·L)	O(n·L)	Se ejecuta cuando el factor de carga superaría 0.75; al duplicar la capacidad, su costo es O(L) amortizado por inserción.
Buscar, Existe	O(L)	O(n·L)	
Eliminar, Editar	O(L)	O(n·L)	Realizan una búsqueda y una remoción sobre la misma cadena.
RegistrarVisita	O(L)	O(n·L)	
ObtenerTodos	O(n + C)	O(n + C)	C = capacidad de la tabla; recorre todas las casillas.

Espacio: O(n + C). La tabla duplica su capacidad al crecer y no la reduce al eliminar, por lo que C puede superar a n.

Trie

Operación	Tiempo	Observación
Insertar	O(L)	Una búsqueda previa O(L) y un recorrido O(L). Es independiente de la cantidad de palabras almacenadas.
Buscar, EmpiezaCon	O(L)	Un recorrido por los caracteres del texto.
Autocompletar	O(L + M·Lmax·(Σ + Lmax))	Tras ubicar el prefijo en O(L), se visitan como máximo los nodos de los caminos hacia las M palabras devueltas. Cada nodo revisa hasta Σ casillas de hijos y construye una cadena de

Operación	Tiempo	Observación
		hasta Lmax caracteres. Con M y Σ constantes, el costo no depende de n.
Autocompletar con prefijo vacío y M = n	O(Σ · N)	N = cantidad de nodos del Trie, con N ≤ suma de las longitudes de los usernames. Es el caso del listado completo de la opción Ver estructuras.
Eliminar	O(L)	Búsqueda, recorrido del camino y poda de ramas.

Espacio: O(Σ · N). Cada nodo reserva un arreglo de 128 referencias, lo que equivale a un costo de memoria elevado por nodo a cambio del acceso directo por carácter.

Union-Find

Operación	Tiempo	Observación
Agregar	O(L)	Alta en IndiceLlaves (esperado) y escritura en los tres arreglos; la duplicación de arreglos es O(1) amortizado.
Buscar, Conectados	O(L + α(n))	Búsqueda de la llave más Raiz con compresión de caminos.
Unir	O(L + α(n))	Unión por rango.
Comunidades	O(n · α(n))	Una operación Raiz por elemento y tres pasadas lineales.
Reiniciar	O(1)	Crea estructuras nuevas de capacidad inicial.

La compresión de caminos combinada con la unión por rango hace que el costo amortizado de Raiz sea O(α(n)). Espacio: O(n).

Grafo

Operación	Tiempo	Observación
AgregarVertice	O(L)	Alta en IndiceLlaves; duplicación de arreglos O(1) amortizada.
AgregarArista	O(L + d(a) + d(b))	La búsqueda de una arista existente recorre la lista de adyacencia; la inserción es O(1).
EliminarArista	O(L + d(a) + d(b))	Busca y remueve en ambas listas.
EliminarVertice	O(L + d(u) + suma de d(v) para cada vecino v de u)	Remueve la arista simétrica en la lista de cada vecino.
Vecinos	O(L + d(u))	Copia la lista a arreglos.

Operación	Tiempo	Observación
BFS	O(V + E)	Cada vértice se encola una vez y cada lista de adyacencia se recorre una vez. La inicialización de los arreglos de control cuesta O(V).
DFS	O(V + E)	Tiempo igual al de BFS. Espacio adicional de pila O(V) en el peor caso, por la recursión.
Dijkstra	O((V + E) log V)	Ver detalle a continuación.
SugerirAmigos	O(V + E + M·V)	Ver detalle a continuación.
ObtenerAristas	O(V + E)	Recorre los vértices activos y sus listas.

Espacio del grafo: O(V + E), porque cada relación se almacena como dos aristas dirigidas (una por sentido).

Dijkstra. La inicialización de dist, previo y listo cuesta O(V). Cada relajación exitosa inserta una entrada en el montículo, de modo que el montículo contiene como máximo 2E + 1 entradas y cada inserción o extracción cuesta O(log E), que es O(log V) porque $E \leq V^2$. Las entradas obsoletas se descartan sin procesar sus aristas (borrado perezoso), por lo que cada arista se examina una sola vez. La reconstrucción del camino busca la arista de cada salto en O(d) y suma, en total, O(E) como máximo. El costo total es O((V + E) log V). El algoritmo se detiene al extraer el destino, lo que en la práctica reduce el trabajo, sin cambiar el peor caso. El costo Costo(afinidad) es O(1) y siempre positivo, condición necesaria para la validez del algoritmo.

Sugerencias de amistad. Con amigos directos, el cálculo de puntajes recorre, para cada amigo, su lista de adyacencia: la suma de esos grados está acotada por O(E). Sin amigos directos, se calcula el grado de cada vértice en O(V). La selección de los M mejores candidatos realiza M pasadas lineales sobre los V vértices, O(M·V). Con M = 5, constante, el costo se reduce a O(V + E).

Servicio de afinidad y persistencia

Operación	Tiempo	Observación
AC.Calcular	O(L + ca · cb)	ca y cb = cantidad de cursos de cada usuario; compara cada curso de uno con los del otro. Como la cantidad de cursos es pequeña, el costo es prácticamente constante.
AC.PesoArista	O(1)	Aritmética con redondeo.
CargarUsuarios (F filas)	O(F · L) esperado	Análisis de cada línea e inserción en la tabla hash.
CargarRelaciones (F filas)	O(F · (L + d))	Dos búsquedas en la tabla hash, el cálculo de afinidad si falta y la

Operación	Tiempo	Observación
		inserción de la arista; d = grado máximo involucrado.
GuardarUsuarios	O(n + C)	Recorre la tabla hash y escribe n filas.
GuardarRelaciones	O(V + E)	Obtiene las aristas y escribe E filas.
Reconstruir	O(n·L + E·(L + α(n)))	Inserta cada username en el Trie, el grafo y Union-Find y une los extremos de las E aristas. Con L acotada equivale a O(n + E).

Operaciones del servicio SNS y de la interfaz

Operación	Caso promedio	Observación
Registrar	O(L)	Cuatro altas de costo O(L) cada una (tabla hash, Trie, grafo y Union-Find); la reversión, si el Trie rechaza la llave, también es O(L).
Buscar (autocompletado)	O(L + M·Lmax·(Σ + Lmax))	Se ejecuta con cada tecla, con M = 7 en la interfaz.
Perfil	O(L)	Dos accesos a la tabla hash (búsqueda y registro de visita).
AgregarRelacion	O(L + d(a) + d(b))	Dos búsquedas, cálculo de afinidad, inserción de la arista y Unir.
CaminoMasFuerte	O((V + E) log V)	Dos verificaciones de existencia, Conectados y Dijkstra; Dijkstra domina. Si los usuarios están en comunidades distintas, la validación evita ejecutar Dijkstra y el costo es O(L + α(n)).
SugerenciasDeAmistad	O(V + E)	Con M constante.
Recorrer	O(V + E)	Ejecuta BFS y DFS.
ObtenerComunidades	O(n · α(n))	
Eliminar (usuario)	O(n·L + E·(L + α(n)))	Domina Reconstruir; las demás bajas cuestan O(L) y O(d + suma de grados de los vecinos).
EliminarRelacion	O(n·L + E·(L + α(n)))	Domina Reconstruir, porque Union-Find no permite deshacer uniones.

Operación	Caso promedio	Observación
EditarUsuario	O(L + d(u) · (L + d(u) + dv))	dv = grado de un vecino; recalcula la afinidad y actualiza la arista de cada vecino.
Repintado de GPanel	O(V' + E')	V' y E' = usuarios y relaciones mostrados. Se ejecuta en cada repintado (clic, cambio de tamaño, selección de camino o de comunidades).

La operación de mayor costo del sistema es la reconstrucción tras una baja, lineal en el tamaño de los datos. Las operaciones frecuentes (búsqueda, perfil, registro y alta de relación) son de costo esperado O(L), prácticamente constante, y las consultas sobre el grafo (recorridos, caminos y sugerencias) crecen linealmente con el tamaño de la red, salvo Dijkstra, que añade el factor logarítmico del montículo.

Justificación de la elección de las estructuras de datos

La selección de cada estructura responde a la operación que el sistema ejecuta con más frecuencia sobre ella y a las restricciones del proyecto, que prohíben usar las colecciones genéricas de la biblioteca estándar (List<T>, LinkedList<T>, Dictionary, HashSet, PriorityQueue, SortedSet) y LINQ. A continuación se justifica cada una.

Tabla Hash con encadenamiento separado

El sistema necesita localizar el perfil completo de un usuario a partir de su Username en cada búsqueda, clic sobre un nodo, registro, relación y baja. La tabla hash ofrece acceso por llave con costo esperado O(L), independiente de la cantidad de usuarios, mientras que un arreglo o una lista enlazada exigiría recorrer los n registros (O(n)) y un árbol de búsqueda balanceado costaría O(log n) y mantendría un orden por llave que el sistema no requiere.

Se eligió el encadenamiento separado porque la baja de un elemento se reduce a quitar un nodo de su cadena, sin marcadores de borrado, y porque tolera cierto grado de colisiones sin degradar el funcionamiento. El factor de carga se limita a 0.75 mediante la duplicación de la capacidad, lo que conserva cadenas cortas y reparte el costo del redimensionamiento entre las inserciones (O(1) amortizado). La función distribuye las llaves mejor que una simple suma de caracteres, lo que reduce las colisiones. La tabla se establece como fuente de verdad porque todas las demás estructuras referencian a los usuarios por Username.

Trie

El buscador debe sugerir usernames mientras se escribe, es decir, resolver consultas por prefijo en cada tecla. En un Trie esa consulta cuesta O(L) para ubicar el prefijo, y el costo posterior depende de la cantidad de sugerencias solicitadas y no de la cantidad de usuarios. Resolver lo mismo con la tabla hash obligaría a recorrer todos los usuarios en cada pulsación (O(n)), y con un arreglo ordenado la búsqueda por prefijo sería eficiente pero la inserción costaría O(n).

El recorrido en orden de código ASCII entrega las sugerencias en orden alfabético sin ordenarlas. El contador Conteo por nodo permite conocer el total de coincidencias y podar ramas vacías al eliminar. Los hijos se guardan en un arreglo de 128 posiciones porque el acceso por carácter es

directo, a costa de más memoria por nodo; este compromiso es aceptable porque los usernames se limitan a ASCII y la cantidad de usuarios de una red universitaria es moderada. El Trie almacena solo llaves y delega el perfil a la tabla hash, lo que evita duplicar datos.

Grafo no dirigido y ponderado con lista de adyacencia

Las relaciones de amistad forman una red en la que el sistema ejecuta recorridos (BFS y DFS), cálculo de caminos y sugerencias por amigos de amigos; el grafo es el modelo natural para estas operaciones. Se eligió la lista de adyacencia frente a la matriz de adyacencia porque una red social es dispersa: la lista ocupa $O(V + E)$ de memoria y permite recorrer los vecinos de un vértice en $O(d)$, mientras que la matriz ocuparía $O(V^2)$ y obligaría a revisar V casillas por vértice.

El grafo es no dirigido porque la amistad es simétrica: cada relación se almacena como dos aristas con la misma afinidad. Es ponderado porque la afinidad (0 a 100) distingue la fuerza de cada relación y alimenta tanto el camino más fuerte como las sugerencias. Los vértices se representan con índices enteros obtenidos de `IndiceLlaves`, lo que permite usar arreglos de enteros y booleanos como estructuras de control en los algoritmos. El borrado lógico (`activo[]`) evita reindexar los vértices al eliminar un usuario.

Union-Find (conjuntos disjuntos)

El sistema necesita saber si dos usuarios pertenecen a la misma comunidad, es decir, si existe algún camino entre ellos, antes de ejecutar Dijkstra, y necesita listar las comunidades. Con Union-Find esa verificación cuesta $O(\alpha(n))$, prácticamente constante, mientras que un BFS por consulta costaría $O(V + E)$. La unión de comunidades al agregar una relación también es $O(\alpha(n))$.

La compresión de caminos y la unión por rango son las dos optimizaciones que producen ese costo casi constante. La limitación de la estructura, que no permite deshacer uniones, se considera aceptable porque las bajas son poco frecuentes frente a las consultas, y se resuelve reconstruyéndola desde el grafo, que conserva la información de las relaciones.

Montículo mínimo (`MinMound`)

Dijkstra necesita extraer en cada paso el vértice pendiente con menor costo acumulado. Con un montículo binario mínimo, la inserción y la extracción cuestan $O(\log m)$; con un arreglo no ordenado, la extracción costaría $O(V)$ y el algoritmo pasaría a $O(V^2)$, y con una lista ordenada, la inserción costaría $O(m)$. El montículo se implementa sobre dos arreglos paralelos (vértices y costos), lo que evita crear un objeto por entrada y aprovecha la contigüidad en memoria. Su uso exige costos positivos, condición que se garantiza con `Costo(afinidad) = 101 − peso`, cuyo valor siempre está entre 1 y 101.

Lista enlazada simple (`LinkedList<T>` propia)

La restricción del proyecto impide usar la lista de la biblioteca estándar, por lo que se implementó una lista enlazada con referencia a la cola. Resulta adecuada para sus dos usos: como cadena de colisiones de la tabla hash, donde las cadenas son cortas y la inserción al final es $O(1)$; y como lista de adyacencia del grafo, donde se recorre secuencialmente y el grado de cada vértice es pequeño en comparación con la cantidad total de usuarios. La búsqueda lineal $O(k)$ no es un inconveniente en ninguno de los dos casos.

Índice de llaves (`IndiceLlaves`)

El grafo y Union-Find trabajan con arreglos indexados por enteros, pero los usuarios se identifican por texto. `IndiceLlaves` traduce cada `Username` a un entero consecutivo con costo esperado O(L), mediante direccionamiento abierto con sondeo lineal. Esta variante se justifica porque la estructura no necesita borrar llaves (los vértices eliminados solo se desactivan), que es precisamente el caso en el que el direccionamiento abierto se complica, y porque almacena enteros en un arreglo contiguo, con buen comportamiento de caché. El factor de ocupación se mantiene en 0.5 o menos para que el sondeo lineal conserve un número bajo de sondeos.

Arreglos con duplicación de capacidad

Los arreglos de `Graph`, `UnionFind`, `MinMound` e `IndiceLlaves` duplican su tamaño al llenarse. La duplicación hace que el costo de agregar un elemento sea O(1) amortizado y evita fijar de antemano un número máximo de usuarios.

Cola (`Queue<int>`) en el recorrido BFS

El BFS requiere una cola FIFO, cuyas operaciones de encolar y desencolar cuestan O(1). Es la única colección de la biblioteca estándar utilizada en la capa de estructuras y se emplea porque el enunciado del proyecto la permite para este recorrido.

Archivos CSV

La persistencia usa archivos de texto plano con un registro por línea, que son legibles y editables sin herramientas especiales y se procesan en una sola pasada. Solo se guardan los datos fuente (usuarios y relaciones); el Trie y Union-Find son índices derivados que se reconstruyen al cargar, lo que evita almacenar información redundante que podría quedar inconsistente.